

PySpark Capstone Project

Telecom Customer Usage and Billing Analytics

Business Scenario

A telecom company wants to analyze:

- Customers
- Mobile Plans
- Usage Records
- Billing
- Payments
- Customer Churn Risk
- City-wise Revenue
- Plan Performance

The goal is to build an end-to-end PySpark ETL pipeline using CSV and JSON data.

Dataset 1: customers.csv

```
%writefile customers.csv
customer_id,customer_name,city,state,age,gender,plan_id,status
101,Rahul Sharma,Hyderabad,Telangana,35,Male,P101,Active
102,Priya Reddy,Bangalore,Karnataka,29,Female,P102,Active
103,Amit Kumar,Mumbai,Maharashtra,42,Male,P103,Inactive
104,Sneha Patel,Chennai,Tamil Nadu,31,Female,P101,Active
105,Farhan Ali,Delhi,Delhi,55,Male,P104,Active
106,Neha Singh,Pune,Maharashtra,38,Female,P102,Active
107,Arjun Verma,Hyderabad,Telangana,26,Male,P103,Inactive
108,Meera Nair,Kochi,Kerala,48,Female,P104,Active
109,Kiran Rao,Bangalore,Karnataka,33,Male,P101,Active
110,Nisha Reddy,Delhi,Delhi,41,Female,P102,Active
111,Ravi Kumar,Mumbai,Maharashtra,45,Male,P105,Active
112,Ayesha Khan,Hyderabad,Telangana,28,Female,,Active
```

Dataset 2: usage.csv

```
%%writefile usage.csv
usage_id,customer_id,usage_month,data_used_gb,call_minutes,sms_count
1001,101,2026-01,45,900,120
1002,102,2026-01,30,600,80
1003,103,2026-01,12,250,40
1004,104,2026-01,55,1100,150
1005,105,2026-01,75,1500,200
1006,106,2026-01,28,500,60
1007,107,2026-01,10,200,20
1008,108,2026-01,80,1600,250
1009,109,2026-01,48,950,100
1010,110,2026-01,32,700,90
1011,120,2026-01,60,1300,140
1012,101,2026-02,50,1000,130
1013,102,2026-02,34,650,85
1014,104,2026-02,58,1200,160
1015,105,2026-02,,1450,210
```

Dataset 3: plans.json

```
%%writefile plans.json
[
  {
    "plan_id": "P101",
    "plan_name": "Smart Basic",
    "monthly_fee": 499,
    "data_limit_gb": 50,
    "features": {
      "unlimited_calls": true,
      "ott_included": false,
      "roaming": "National"
    }
  },
  {
```

```
"plan_id": "P102",
"plan_name": "Smart Plus",
"monthly_fee": 799,
"data_limit_gb": 75,
"features": {
  "unlimited_calls": true,
  "ott_included": true,
  "roaming": "National"
},
{
  "plan_id": "P103",
  "plan_name": "Budget Saver",
  "monthly_fee": 299,
  "data_limit_gb": 25,
  "features": {
    "unlimited_calls": false,
    "ott_included": false,
    "roaming": null
  },
{
  "plan_id": "P104",
  "plan_name": "Premium Max",
  "monthly_fee": 1199,
  "data_limit_gb": 100,
  "features": {
    "unlimited_calls": true,
    "ott_included": true,
    "roaming": "International"
  }
}
]
```

Dataset 4: payments.csv

```
%%writefile payments.csv
payment_id,customer_id,bill_month,amount_paid,payment_mode,payment_status
5001,101,2026-01,499,UPI,Success
5002,102,2026-01,799,Card,Success
5003,103,2026-01,299,Cash,Failed
```

5004,104,2026-01,499,UPI,Success
5005,105,2026-01,1199,Card,Success
5006,106,2026-01,799,UPI,Success
5007,107,2026-01,299,Cash,Pending
5008,108,2026-01,1199,Card,Success
5009,109,2026-01,499,UPI,Success
5010,110,2026-01,799,UPI,Success
5011,112,2026-01,,UPI,Success
5012,101,2026-02,499,Card,Success
5013,102,2026-02,799,UPI,Success
5014,104,2026-02,499,UPI,Success
5015,105,2026-02,1199,,Pending

Capstone Tasks

Part 1: Ingestion

1. Read customers.csv.
 2. Read usage.csv.
 3. Read payments.csv.
 4. Read plans.json using multiline option.
 5. Display schema of all datasets.
 6. Count records in all datasets.
 7. Save raw datasets as Bronze Parquet.
-

Part 2: Data Cleaning

8. Find customers with missing plan_id.
9. Find usage records with missing data_used_gb.
10. Find payments with missing amount_paid.
11. Find payments with missing payment_mode.
12. Fill missing data_used_gb with 0.
13. Fill missing amount_paid with 0.
14. Fill missing payment_mode with Not Provided.
15. Fill missing plan_id with UNKNOWN.
16. Create data_quality_status column in each dataset.
17. Save cleaned datasets as Silver Parquet.

Part 3: JSON Flattening

18. Flatten plans.json.
 19. Extract unlimited_calls.
 20. Extract ott_included.
 21. Extract roaming.
 22. Fill missing roaming with Not Available.
 23. Save flattened plans as Parquet.
-

Part 4: Joins

24. Join customers with plans.
 25. Join customers with usage.
 26. Join customers with payments.
 27. Create complete customer usage billing DataFrame.
 28. Find customers with invalid plan IDs.
 29. Find usage records without matching customer.
 30. Find payments without matching customer.
-

Part 5: Transformations

31. Create usage_category:
data_used_gb >= 70 -> Heavy User
data_used_gb >= 30 -> Medium User
Else -> Low User
32. Create payment_category:
amount_paid >= 1000 -> High Payment
amount_paid >= 500 -> Medium Payment
Else -> Low Payment
33. Create churn_risk:
status = Inactive OR payment_status != Success -> High Risk
data_used_gb < 15 -> Medium Risk
Else -> Low Risk

34. Create over_usage_gb:
data_used_gb - data_limit_gb
 35. Create over_usage_flag:
over_usage_gb > 0 -> Yes
Else -> No
-

Part 6: Aggregations

36. Count customers by city.
 37. Count customers by state.
 38. Count customers by plan.
 39. Count customers by usage_category.
 40. Count customers by churn_risk.
 41. Calculate total data usage by plan.
 42. Calculate average data usage by plan.
 43. Calculate total call minutes by city.
 44. Calculate total SMS count by state.
 45. Calculate total successful revenue.
 46. Calculate revenue by city.
 47. Calculate revenue by plan.
 48. Calculate revenue by payment_mode.
 49. Find plan with highest revenue.
 50. Find city with highest revenue.
-

Part 7: Window Functions

51. Rank customers by data usage.
 52. Rank customers by amount paid.
 53. Find top 3 data users.
 54. Find top 3 revenue customers.
 55. Find top customer by city.
 56. Find top customer by plan.
 57. Calculate running total revenue by month.
 58. Use lag to compare current month and previous month data usage.
 59. Use lead to show next month usage.
 60. Find customers whose usage increased month over month.
-

Part 8: Spark SQL

61. Create temp views for customers, usage, payments and plans.
 62. Write SQL query to display active customers.
 63. Write SQL query to count customers by city.
 64. Write SQL query to calculate revenue by plan.
 65. Write SQL query to find heavy users.
 66. Write SQL query to find high churn risk customers.
 67. Write SQL query to find invalid plan customers.
 68. Write SQL query to find failed or pending payments.
 69. Write SQL query to find top 5 customers by data usage.
 70. Write SQL query to find revenue by payment mode.
-

Part 9: Full Refresh and Incremental Load

71. Write final Gold output using overwrite mode.
 72. Partition Gold output by usage_month.
 73. Create a new incremental usage file for 2026-03.
 74. Read incremental file.
 75. Append new usage data to existing Silver usage Parquet.
 76. Recalculate customer usage summary.
 77. Recreate Gold output partitioned by usage_month.
 78. Compare record count before and after incremental load.
-

Part 10: Final Gold Reports

79. Customer Usage Summary
 - customer_id
 - customer_name
 - city
 - plan_name
 - usage_month
 - data_used_gb
 - data_limit_gb
 - over_usage_flag
 - amount_paid
 - payment_status

churn_risk

80. Plan Performance Report

plan_name

total_customers

total_data_usage

average_data_usage

total_revenue

81. City Revenue Report

city

total_customers

total_revenue

average_payment

82. Churn Risk Report

customer_id

customer_name

city

plan_name

payment_status

status

churn_risk

83. Over Usage Report

customer_id

customer_name

plan_name

data_used_gb

data_limit_gb

over_usage_gb
